Universidad Tecnológica Nacional Facultad Regional Córdoba

Ingeniería en Sistemas de

Facultad Regional Córdoba

Ingeniería y Calidad de Software

Trabajo Práctico N° 6: TDD

Documento de justificación de decisiones de diseño

User Story: Comprar entradas (EcoHarmony Park)

Curso: 4K1

Grupo: N° 7

Docentes:

- Ing. Silvia Judith Meles
- Ing. Valeria Rosa Abdala
- Juan Salvador Barbera

Integrantes:

N°	Apellido y Nombre	Legajo
1	Boaglio, Tomás	94597
2	Dagnino Dailly, Facundo	94307
3	Demaría, Francisco	96142
4	Farias, Tomás	94566
5	Farnochi Lahís, Sebastián	89682
6	Gardella Jornet, Tomás	402552
7	Moyano Aberastegui, Juan Ignacio	94410
8	Navarro, Luca	400474
9	Peralta Conci, Santiago	402548
10	Pezoa Gechelin, Valentino	400056
11	Pfening, Facundo	404613
12	Rossi, Franco	400050
13	Santillán, Tomás	400346
14	Scatena, Marco	400745
15	Vanega, Ayrton	95386

ÍNDICE

Introducción....3
Justificación de la arquitectura elegida....3
1. Capa de Dominio (el foco del TDD)....4
2. Capa de Aplicación....4
3. Capa de Presentación (adaptador web)....4
4. Capa de Verificación....5
Modelado y decisiones de diseño....5
1. Dominio con objetos inmutables y enumeraciones....5
2. Errores de validación con campo y mensaje....5
3. Reglas de negocio como constantes con nombre....6
4. Reloj inyectado y una sola regla de disponibilidad....6
5. Límite de 10 entradas por usuario y fecha....7
6. Estados de la compra y transiciones en una función pura....7
7. Persistencia detrás de un contrato de repositorio....7
8. Pasarela de pago simulada y datos de tarjeta....8
9. Mail y QR desacoplados del dominio....9
10. Inyección de dependencias y usuario de sesión....9
11. Frontend responsive sin frameworks....9
Aplicación del ciclo Red-Green-Refactor....11
Forma de trabajo....11
Fase Red....11
Fase Green....12
Fase Refactor....13
Cómo funcionan las pruebas....13
Trazabilidad con los criterios de aceptación....14

Introducción

Este documento justifica las decisiones de diseño tomadas al implementar la user story **"Comprar entradas"** de EcoHarmony Park. La implementación se realizó aplicando Desarrollo Conducido por Pruebas (TDD) con el ciclo **Red-Green-Refactor**: cada regla de negocio se escribió primero como una prueba automatizada que fallaba, después se agregó el código mínimo para que pasara y, con la suite en verde, se mejoró el diseño sin cambiar el comportamiento.

Se construyó una aplicación web responsive con backend en **Python** (Flask), persistencia en **SQLite** y pruebas automatizadas con **pytest**. El código respeta la guía de estilo **PEP8**, verificada con `flake8` (líneas de hasta 79 caracteres), y usa nombres de dominio en español (D7).

La funcionalidad permite que el usuario de la sesión elija la fecha de visita en un calendario que solo habilita los días en que el parque abre, cargue hasta 10 visitantes con su edad y tipo de pase (Común o VIP), vea el total con los descuentos por edad y elija la forma de pago. En **efectivo**, la compra queda pendiente de pago en boletería; con **tarjeta**, se pasa por una **pasarela de pago simulada** con el aspecto de Mercado Pago. Al confirmarse la compra se informan la cantidad de entradas y la fecha, y se envía un **mail de confirmación con un código QR** (envío real por SMTP si se configura la cuenta de Gmail en el archivo `.env`; si no, se simula).

Las reglas de negocio salen de la user story y sus criterios de aceptación (CA1 a CA9), de las FAQs respondidas por los PO y de los supuestos S1 a S10, consultados con el PO y respondidos el 20/10/2026. Cada decisión de este documento se identifica con su número en el registro de decisiones del repositorio (D1 a D52, en `docs/Decisiones_de_Diseno.md`), y cada caso de prueba con su ID en `docs/Casos_de_Prueba.md` (por ejemplo `PRE-05` o `TAR-12`).

Referencias:

CA n: criterio de aceptación de la US.

FAQ n: respuesta de los PO en el documento de FAQs.

S n: supuesto consultado con el PO (S1 a S10, respondidos el 20/10/2026) o decisión del grupo sobre la pasarela (S11 a S15). Están en Casos_de_Prueba.md.

D n: fila del registro completo en Decisiones_de_Diseno.md.

Link del repositorio: https://github.com/frandemariaUTN/isw_repo_grupo_7

Justificación de la arquitectura elegida

La arquitectura del sistema se estructuró en cinco capas claramente delimitadas (Dominio, Aplicación, presentacion, adaptadores y Verificación) para maximizar la testeabilidad, aislar las reglas de negocio y facilitar la aplicación rigurosa del ciclo de Desarrollo Conducido por Pruebas (TDD).

Capa	Carpeta	Contenido	Cómo se prueba
Dominio	src/ecoharmony/dominio/	Precios, edades, pases, calendario, fechas, cantidades, pagos, compras, tarjetas, armado del mail y pasarela simulada	176 pruebas unitarias, sin I/O
Aplicación	src/ecoharmony/aplicacion/	ServicioDeCompra: comprar, pagar y cancelar	Pruebas de integración (SRV)
Adaptadores	persistencia/ y notificaciones/	RepositorioSQLite, NotificadorSMTP, NotificadorSimulado	Pruebas de integración (BD, MAIL)
Presentación	src/ecoharmony/web/	App Flask, plantillas HTML, CSS y JavaScript	Integración (API) y sistema (SIS)
Verificación	tests/	unit/, integration/ y system/	272 pruebas automáticas + 8 de sistema

1. Capa de Dominio (el foco del TDD)

- **Aislamiento tecnológico:** el dominio es Python puro, sin Flask, sin base de datos y sin llamadas a internet. Por eso las 272 pruebas unitarias y de integración se ejecutan en menos de un segundo, y el ciclo Red-Green-Refactor se puede repetir muchas veces sin esperar.
- **Dependencias externas detrás de interfaces** (D4, D6): el repositorio, la pasarela de pago y el envío del mail tienen una implementación real o simulada y una en memoria para los tests. El dominio no cambia si se cambia la base de datos o el servidor de mail.

2. Capa de Aplicación

- `ServicioDeCompra` coordina el caso de uso: crea la compra con las reglas del dominio, la guarda, abre la sesión de pago si corresponde y envía el mail. Recibe el repositorio, la pasarela y el notificador por constructor y devuelve un `ResultadoDeCompra` (compra, URL de pago, destinatario del mail, mensaje final, motivo de rechazo y aviso si el mail falló) (D39).

3. Capa de Presentación (adaptador web)

- **Controlador delgado:** `web/app.py` solo traduce HTTP a llamadas al servicio y el resultado a pantallas o JSON. La app se arma con la función `crear_app`, que recibe sus dependencias (D41).
- **Rutas:** `GET /` (pantalla de compra), `GET /api/dias-no-disponibles` y `GET /api/precios` (datos para el calendario y el total en vivo), `POST /api/compras` (crear la compra), `GET /pasarela/<token>`, `POST /pasarela/<token>/pagar` y `POST /pasarela/<token>/cancelar` (pasarela simulada) y `GET /compras/<numero>/resultado` (pantalla final).

- **Códigos HTTP:** `201` cuando la compra se crea; `400` con la lista de errores (campo y mensaje) cuando los datos no son válidos, o con la pantalla de la pasarela marcando los campos de la tarjeta; `303` para volver de la pasarela a la pantalla de resultado; y `404` para una sesión de pago inexistente o una orden de otro usuario.
- **Frontend:** HTML, CSS y JavaScript sin frameworks, servido por Flask (D43). La compra se envía como JSON y la pasarela usa formularios HTML comunes con redirecciones (D42).

4. Capa de Verificación

- **Tres niveles de prueba:** unitarias para las reglas del dominio (176), de integración para las piezas conectadas: SQLite, pasarela, mail, servicio y API (96), y de sistema en el navegador con Playwright y el Microsoft Edge instalado (8) (D46). Además hay una prueba manual que envía un mail real (`pytest -m smtp_real`), excluida de la corrida por defecto.
- Cada prueba lleva el ID de su caso en `docs/Casos_de_Prueba.md`, así el reporte de pytest muestra qué regla se está verificando.

Modelado y decisiones de diseño

1. Dominio con objetos inmutables y enumeraciones

- Las entidades del dominio son `@dataclass(frozen=True)`: `Visitante`, `Compra`, `Usuario`, `TarjetaValidada`, `SesionDePago` y `ResultadoDeCompra` (D12, D27). Un visitante o una compra no cambian después de creados y se comparan por contenido.
- Un cambio de estado produce un objeto nuevo con `dataclasses.replace`, sin modificar el original. Así lo hacen `ServicioDeCompra._aplicar` al pasar una compra a PAGADA o FALLIDA y `PasarelaSimulada._actualizar` al cerrar una sesión de pago. Esto hace que las pruebas sean determinísticas: ningún objeto cambia a escondidas durante la ejecución.
- Los valores cerrados son enumeraciones (Enum): `Pase` (COMUN, VIP), `FormaDePago` (EFECTIVO, TARJETA), `EstadoCompra` (PENDIENTE_BOLETERIA, PENDIENTE_PAGO_ONLINE, PAGADA, FALLIDA), `ResultadoDePago` (APROBADO, RECHAZADO, PENDIENTE) y `EstadoSesion` (ABIERTA, CERRADA, CANCELADA). El texto que llega del formulario se normaliza y se convierte al enum, y cualquier otro valor se rechaza.

2. Errores de validación con campo y mensaje

- Cada validación lanza `ErrorDeValidacion`(campo, mensaje) (D14). El campo identifica qué dato está mal (fecha_visita, cantidad, forma_de_pago), y para los visitantes incluye la posición empezando en 1, como la ve la persona que carga el formulario: `visitantes.2.edad` (D15).

- `crear_compra` no se detiene en el primer error: acumula todos con la función auxiliar `_intentar` y los lanza juntos en `ErroresDeValidacion` (D21). La API los devuelve con código 400 y el formulario marca todos los campos incorrectos de una vez. La falta de sesión de usuario, en cambio, se rechaza de inmediato.
- Hay dos errores más para las piezas externas: `ErrorDeNotificacion` cuando no se pudo enviar el mail y `ErrorDePasarela` cuando la sesión de pago no está disponible.
- Estos errores permitieron escribir los tests en la fase Red esperando un error exacto: por ejemplo, `with pytest.raises(ErrorDeValidacion)` y luego verificar que `error.value.campo` sea "cantidad".

3. Reglas de negocio como constantes con nombre

Los valores que definió el PO están en constantes con nombre, en un solo lugar (D10). Si el PO cambia un valor, se cambia una línea:

Constante	Valor	Origen
PRECIO_POR_PASE	Común $10.000, VIP $20.000	FAQ 6
EDAD_MAX_GRATIS	3 años (no pagan)	FAQ 7
EDAD_MAX_MENOR_CON_DESCUENTO	15 años (50%)	FAQ 7
EDAD_MIN_MAYOR_CON_DESCUENTO	60 años (50%)	FAQ 7
EDAD_MAXIMA	120 años	S6 (D19)
LIMITE_POR_FECHA	10 entradas	FAQ 27, CA7
LUNES y FERIADOS_CERRADO	Lunes, 25/12 y 1/1	FAQ 1, CA5
HORA_LIMITE_COMPRA	17:30	FAQ 26
LARGO_DEL_NUMERO	16 dígitos de tarjeta	D33

- **Montos como enteros en pesos** (D8): los precios del parque no tienen centavos y así se evitan los errores de redondeo de `float`. El 50% se calcula con `precio // 2`, que es exacto porque los precios son pares (D11).
- **La tabla de precios entra por parámetro** en `calcular_precio` y `calcular_total`, con la tabla oficial como valor por defecto (D13). Los tests `PRE-16` prueban otros precios sin tocar el código.
- La pantalla obtiene los precios y las edades límite de `GET /api/precios`, que los lee de estas mismas constantes, para recalcular el total en vivo.

4. Reloj inyectado y una sola regla de disponibilidad

- Las reglas dependen de la hora actual (compra para hoy hasta las 17:30, hasta un mes adelante), así que **la hora no se lee dentro del dominio: entra como parámetro** `ahora` (D5). Los tests usan siempre un reloj fijo, el miércoles 14/10/2026 a las 10:00, y nunca la fecha real.

- `ahora` debe tener zona horaria (si no, `a_hora_local` lanza `ValueError`) y se convierte a la hora del parque, `ZONA_PARQUE`, que es UTC-3 fijo: Argentina no tiene horario de verano, y así no se depende de la base de zonas horarias, que en Windows requiere instalar un paquete extra (D16).
- **Una sola función decide si un día está disponible** (D18): `motivo_no_disponible(fecha, ahora)` devuelve el motivo (fecha pasada, más de un mes adelante, parque cerrado, o compra para hoy después de las 17:30) o `None`. La usa `validar_fecha_visita` para validar la compra y `dias_no_disponibles` para la ruta `/api/dias-no-disponibles`, que deshabilita esos días en el calendario propio de la pantalla (D45). Así la pantalla y el servidor no pueden contradecirse.

5. Límite de 10 entradas por usuario y fecha

- `validar_cantidad` acepta de 1 a `LIMITE_POR_FECHA` entradas por compra, y `crear_compra` controla además que la cantidad coincida con los visitantes cargados.
- El límite se suma entre compras (D24, FAQ 27): `_validar_limite` pide al repositorio las compras del usuario para esa fecha (`compras_de`) y `contar_entradas` suma sus entradas **sin contar las compras `FALLIDA`**, que liberan el cupo (S4). Las entradas gratuitas y las compras pendientes sí cuentan (S2, S3). El mensaje informa cuántas se pueden comprar todavía.
- En la pantalla, el botón para agregar visitantes se deshabilita al llegar a 10 y se muestra un aviso (prueba de sistema `SIS-04`).

6. Estados de la compra y transiciones en una función pura

- `estado_inicial` define el estado al crear la compra: en efectivo queda `PENDIENTE_BOLETERIA`; con tarjeta queda `PENDIENTE_PAGO_ONLINE` **aunque el total sea $0**, porque el PO pidió que toda compra con tarjeta pase por la pasarela (S8, D50).
- `aplicar_resultado_de_pago(estado, resultado)` es una función pura (D22): solo cambia una compra `PENDIENTE_PAGO_ONLINE` (aprobado pasa a `PAGADA`, rechazado a `FALLIDA`) y en cualquier otro caso devuelve el estado como estaba. Por eso es idempotente: si el resultado del pago llega dos veces (doble clic o recarga), el estado no cambia.
- Cancelar en la pasarela se trata como un rechazo: la compra queda `FALLIDA`, no se reintenta y se informa con un pop-up al volver a la app, como pidió el PO (S4, S13, D51).

7. Persistencia detrás de un contrato de repositorio

- `RepositorioEnMemoria` (en el dominio) define el contrato: `siguiente_numero_de_orden`, `guardar`, `buscar` y `compras_de` (D23). `RepositorioSQLite` cumple el mismo contrato, y las pruebas de contrato `BD-12` se

corren contra los dos con un fixture parametrizado (`BD-12-memoria` y `BD-12-sqlite`). Así se garantiza que son intercambiables.

- Se eligió **SQLite** (`sqlite3`, de la biblioteca estándar de Python) en vez de archivos CSV (D35): no requiere instalar nada, resuelve con una consulta el límite de 10 por fecha y con transacciones los guardados a medias. La base tiene tres tablas: `secuencia`, `compras` y `visitantes`.
- Cada operación abre una conexión dentro de una transacción, y el número de orden sale de la tabla `secuencia` con `UPDATE ... RETURNING` (D37). Si algo falla no queda una compra a medias, y dos pedidos simultáneos nunca reciben el mismo número de orden.
- **Los datos de la tarjeta nunca se guardan** (D32, S15): ninguna tabla tiene columnas para la tarjeta.

8. Pasarela de pago simulada y datos de tarjeta

- El CA4 pide redirigir a Mercado Pago y la FAQ 4 autoriza simularlo, así que se hizo una pasarela propia con su aspecto, sin credenciales ni servicios externos (D28). `PasarelaSimulada` está en el dominio porque reemplaza a un servicio externo y se prueba igual que él (casos `PSM`).
- Cada pago tiene una **sesión con un token aleatorio** (`secrets.token_urlsafe(32)`): la pasarela no necesita saber quién es el usuario y nadie puede adivinar la sesión de otro (D29). Las sesiones viven en memoria, como las de un servicio externo; la app solo guarda la compra y su estado (D38).
- **El resultado se elige con el nombre del titular**, como en las tarjetas de prueba de Mercado Pago (D30, S12): `OTHE`, `FUND`, `SECU` y `EXPI` rechazan, `CONT` queda pendiente y cualquier otro titular válido aprueba. Están en el diccionario `RESULTADOS_POR_TITULAR`. Así se pueden demostrar los rechazos sin pantallas extra.
- **Todo se valida antes de ir a la pasarela** (D31): `ServicioDeCompra.comprar` primero crea la compra con todas sus validaciones y recién después abre la sesión de pago. La pasarela solo valida la tarjeta: número de 16 dígitos que cumpla el algoritmo de Luhn, con la marca detectada por el primer dígito (D33); vencimiento; código de seguridad; titular con letras, acentos, ñ, espacios, apóstrofos o guiones (D34); y DNI de 7 u 8 dígitos (FAQ 32).
- Después de validar solo se conserva `TarjetaValidada` (marca, últimos 4 dígitos y titular) (D32). Si hay errores, la pantalla vuelve a mostrar el titular, el vencimiento y el DNI, pero nunca el número ni el código de seguridad.
- **Doble clic seguro** (D40): si se intenta pagar una sesión ya resuelta, `ServicioDeCompra.pagar` devuelve el resultado anterior en vez de un error, sin cobrar ni enviar el mail dos veces.

9. Mail y QR desacoplados del dominio

- El dominio arma el mail y un adaptador lo envía (D26). `armar_mail(compra)` devuelve un objeto `Mail` con el destinatario, el asunto, el cuerpo (número de orden, fecha, detalle de visitantes con subtotales y total, según la FAQ 18) y el texto del QR. `contenido_qr` arma ese texto legible: orden, fecha, total y entradas agrupadas por pase y edad (menor, adulto, adulto mayor) (D47, S5).
- Hay dos notificadores con la misma interfaz: `NotificadorSimulado`, que registra el mail sin enviarlo, y `NotificadorSMTP`, que lo envía con `smtplib` y STARTTLS (Gmail con contraseña de aplicación), con versión de texto, versión HTML con el QR incrustado y el QR adjunto, dibujado como PNG con la librería `segno` (D49). Así el dominio no depende de ninguna librería externa.
- `elegir_notificador` usa el envío real solo si el archivo `.env` tiene la cuenta; si no, usa el simulado, que es el que usan los tests (D36). El `.env` nunca se sube a git.
- El mail se envía al **confirmar** la compra (S9): en efectivo al crearla y con tarjeta cuando se aprueba el pago. Esa decisión está en un solo lugar, `ServicioDeCompra._confirmar`.
- **Si el mail falla, la compra se mantiene** (D49, S10): `_confirmar` captura `ErrorDeNotificacion` y devuelve el resultado con el aviso `AVISO_SIN_MAIL`, que la pantalla final muestra en lugar de la dirección (caso `SRV-15`).

10. Inyección de dependencias y usuario de sesión

- `crear_app` recibe el repositorio, la pasarela, el notificador, el reloj y el usuario (D41). Sin argumentos usa SQLite en `data/ecoharmony.db`, la pasarela simulada, `elegir_notificador`, la hora real y el usuario de sesión; así la levanta `run.py`. Los tests de la API y del sistema la arman con el repositorio en memoria, el notificador simulado y el reloj fijo, sin cambiar código.
- El usuario de sesión es fijo ("Salva", `usuario_mock`) y se inyecta en la app (D52): la FAQ 19 permite mockear el login, y así se cumple el CA9 (solo usuarios registrados) sin construir registro ni inicio de sesión.

11. Frontend responsive sin frameworks

- El PO pide una app mobile y acepta una web responsive (FAQ 24), así que la pantalla se diseñó primero para un celular de 390 px de ancho y se adapta a la computadora (D43). La prueba `SIS-08` verifica que en una pantalla de 375×667 no haya scroll horizontal y que los botones midan al menos 44 px.
- Se usó HTML, CSS y JavaScript sin frameworks: alcanza para una sola pantalla de compra, no agrega dependencias y Flask la sirve directo (D43).
- Se usa la paleta de la FAQ 3 (`#386641`, `#6A994E`, `#A7C957`, `#F2E8CF`, `#BC4749`) con la fuente Lato, una rana de ojos rojos como mascota y el logo aprobado por el PO (FAQ 17). Las animaciones son cortas y se desactivan si el sistema pide reducir el movimiento (D44).

- El calendario es propio porque el selector de fecha nativo no permite deshabilitar días sueltos, y la FAQ 2 pide que los días cerrados no se puedan elegir (D45).

Aplicación del ciclo Red-Green-Refactor

Forma de trabajo

Los casos de prueba se definieron antes de programar en `docs/Casos_de_Prueba.md`, cada uno con su ID, resultado esperado y origen (criterio de aceptación, FAQ o supuesto). Se avanzó de menor a mayor complejidad, empezando por el dominio puro: precios y descuentos (`PRE`), edad y pase (`EDA`, `PAS`), calendario y fechas (`DIA`, `FEC`), cantidad y límite (`CAN`), pagos, orden, usuario y mail (`PAG`, `ORD`, `USR`, `MAI`), tarjeta (`TAR`), base de datos (`BD`), pasarela y envío del mail (`PSM`, `MAIL`), servicio (`SRV`), API (`API`) y sistema (`SIS`).

Los primeros casos se trabajaron test por test. Para avanzar más rápido, el resto se trabajó **por grupos de casos** (D20): un commit RED con todos los tests del grupo y uno o varios commits GREEN con el código. Se pierde el detalle test por test, pero la evidencia Red-Green de cada grupo queda en el historial de git: antes de cada commit RED se verificó que los tests fallaran por el motivo esperado, y en cada GREEN pasa toda la suite. Los refactors van en commits aparte. Los mensajes siguen el nombrado del README del repositorio (D9):

Fase	Ejemplo de commit
Red	`feat: [RED] tests PRE-05 a PRE-07 menores de 15 años pagan mitad`
Green	`feat: [GREEN] PRE-05 a PRE-07 menores de 15 años pagan mitad`
Refactor	`refactor: constantes con nombre y funcion para descuento por edad`
Test que pasa de entrada	`feat: tests PRE-11 a PRE-13 descuentos por edad sobre pase VIP`

Fase Red

En Red escribimos las pruebas antes de implementar el comportamiento. Una prueba roja tiene que fallar **por la razón correcta**: porque el comportamiento todavía no existe, no por un error de tipeo.

El primer test (`PRE-01`, commit `9727e44`) importaba el módulo `ecoharmony.dominio.pases`, que todavía no existía, así que pytest no pudo ni cargarlo: `ModuleNotFoundError`. A diferencia de dejar clases vacías que lancen `NotImplementedError`, no escribimos código antes del test: el primer fallo es justamente que no hay código. Lo mismo pasa en cada RED que abre un módulo nuevo: el de edades, pases y calendario (`e608b91`), el de tarjetas (`4ec4255`), el repositorio SQLite (`a8d676f`, falta `ecoharmony.persistencia`), el servicio de compra (`9cd8c26`, falta `ecoharmony.aplicacion`) y la capa web.

Cuando el módulo ya existe, el RED muestra tests que fallan junto a otros que siguen pasando. Por ejemplo, en `642c110` (menores de 15 años pagan mitad) el resultado fue **4 fallidos y 3 aprobados**: fallaban `PRE-03` y `PRE-04` (menores de 3 años gratis, del RED

anterior) y `PRE-05` y `PRE-06`, mientras `PRE-01`, `PRE-02` y `PRE-07` (16 años, precio completo) ya pasaban.

Red no significa que todo el proyecto esté roto. Cuando el PO cambió el supuesto S8 (una compra de $0 con tarjeta pasa igual por la pasarela), el RED `301a729` quedó con **3 fallidos y 249 aprobados**: solo fallaban los tres casos nuevos (`PAG-13`, `PSM-18` y `SRV-13`). Lo mismo con el aviso del mail en la pantalla final (`b2f93ed`): **3 fallidos y 269 aprobados**.

Resultado de pytest en algunos commits RED (sin las pruebas de sistema):

Commit	Casos	Resultado
`9727e44`	PRE-01 precio de pase común para adulto	1 error: no existe `dominio.pases`
`642c110`	PRE-05 a PRE-07 menores de 15 pagan mitad	4 fallidos, 3 aprobados
`e608b91`	EDA, PAS, DIA y FEC	5 errores: no existen los módulos
`4ec4255`	TAR-01 a TAR-25 validación de tarjeta	1 error: no existe el módulo
`a8d676f`	BD-01 a BD-15 repositorio SQLite	1 error: no existe `persistencia`
`9cd8c26`	SRV-01 a SRV-14 servicio de compra	1 error: no existe `aplicacion`
`301a729`	PAG-13, PSM-18 y SRV-13 compra de $0 con tarjeta	3 fallidos, 249 aprobados
`a5032da`	SRV-15 si falla el mail la compra se mantiene	1 error: no existe `ErrorDeNotificacion`
`b2f93ed`	API-19, API-23, API-24 aviso del mail	3 fallidos, 269 aprobados

Fase Green

En Green implementamos el código mínimo para que pasen las pruebas, sin perfeccionar el diseño antes de tiempo. El primer GREEN (`7514431`) fue casi trivial: `Pase` tenía solo el valor `COMUN` y `calcular_precio` devolvía siempre `10000`. Eso alcanzaba para `PRE-01`, y fue el siguiente test (`PRE-02`, pase VIP a $20.000) el que obligó a generalizarlo con la tabla de precios.

Los grupos grandes se pusieron en verde por partes, un commit GREEN por subgrupo. Por ejemplo, después del RED `e608b91` vinieron los GREEN de edad, pase, construcción de visitantes, calendario y fecha de visita, hasta llegar a 71 pruebas aprobadas. La suite fue creciendo así (pruebas unitarias y de integración aprobadas en cada GREEN):

Commit	Grupo que pasó a verde	Pruebas aprobadas
`7514431`	PRE-01	1
`e1efc7e`	PRE-05 a PRE-07	7
`dfc03a4`	EDA, PAS, DIA y FEC	71
`9c85fc0`	CAN, PAG, USR, ORD y MAI	127
`5b9b65f`	TAR	170
`6fac112`	BD	192

Commit	Grupo que pasó a verde	Pruebas aprobadas
`4e0a072`	PSM y MAIL (simulado)	214
`d401d86`	SRV	229
`bc077aa`	API	252
`788e222`	MAIL-04 a MAIL-11 (envío por SMTP)	269
`87895e3`	SRV-15	270
`f1cb45a`	API-19, API-23 y API-24	272

Las 8 pruebas de sistema (`SIS-01` a `SIS-08`) se escribieron sobre el frontend ya construido, por eso se commitearon como `feat: test` sin marca de fase, como indica la regla del equipo para los tests que no pasan por Red. Los cambios posteriores que pidió el PO sobre esas pantallas (pop-up de rechazo, QR, aviso del mail) sí pasaron por Red y Green junto con las pruebas de sistema afectadas.

Fase Refactor

En Refactor mejoramos la estructura interna sin cambiar el comportamiento observable, con todas las pruebas en verde antes y después:

- **Constantes con nombre para los descuentos** (`5f6a2b1`, D10): `calcular_precio` comparaba la edad con números sueltos (`3`, `15`, `60`). Se reemplazaron por `EDAD_MAX_GRATIS`, `EDAD_MAX_MENOR_CON_DESCUENTO` y `EDAD_MIN_MAYOR_CON_DESCUENTO`, y las dos condiciones del 50% se unieron en la función `tiene_descuento_por_edad`. La suite tenía 13 pruebas aprobadas antes y después.
- **Conversión a entero compartida** (`9d1e373`, D25): la validación de edad y la de cantidad repetían la misma lógica para convertir lo que llega del formulario a un número entero. Se extrajo a `a_entero` en `conversiones.py`, y `edades.py` pasó a usarla (14 líneas menos). La suite tenía 127 pruebas aprobadas antes y después.

Al cerrar el último ciclo, la suite completa queda así: **272 pruebas aprobadas** (176 unitarias y 96 de integración) en menos de un segundo, más las **8 pruebas de sistema** en el navegador y la prueba manual de envío real del mail. `flake8 src tests` no informa ningún error de estilo.

Cómo funcionan las pruebas

Usamos **pytest**, que descubre automáticamente los archivos y funciones cuyo nombre empieza con `test_`. La configuración está en `pytest.ini`: indica dónde está el código (`src`), dónde están las pruebas (`tests`) y excluye por defecto la prueba que envía un mail real (marca `smtp_real`). Las pruebas siguen la estructura **Arrange, Act, Assert**: se preparan los datos, se ejecuta la operación y se verifica el resultado.

- **IDs de los casos en el reporte:** los tests se parametrizan con `pytest.param(..., id="PRE-05")`, así cada combinación aparece como un caso independiente con su

ID, y se puede correr uno solo con `pytest -k PRE-05`. Una sola función, por ejemplo, prueba las edades 4, 15 y 16 (`PRE-05`, `PRE-06` y `PRE-07`).

- **Fixtures compartidas** (`tests/unit/conftest.py`): `ahora` (el reloj fijo, 14/10/2026 10:00 en UTC-3), `usuario` (el usuario de sesión), `repositorio` (uno en memoria nuevo para cada test), `datos_validos` (una compra válida), `comprar` (crea una compra cambiando solo los datos que el test indica) y `compra_previa` (para probar el límite de 10 con compras anteriores). pytest las crea para cada prueba, así los casos son independientes y repetibles.
- **Errores esperados:** en los casos inválidos se usa `with pytest.raises(ErrorDeValidacion)` y se verifica el campo informado. Si no se lanza el error, o se lanza otro, el test falla.
- **Pruebas de contrato:** el fixture `repositorio` de `test_repositorio_sqlite.py` está parametrizado con `memoria` y `sqlite`, así las mismas pruebas (`BD-12`) se corren contra los dos repositorios.
- **Envío del mail sin servidor real:** `test_notificador_smtp.py` reemplaza la conexión SMTP por un `ServidorFalso` que registra los pasos (conexión, STARTTLS, inicio de sesión, envío), y así se verifica el mail armado sin mandar nada a internet.
- **API como caja negra:** `test_api.py` usa el cliente de pruebas de Flask para enviar pedidos HTTP y comprobar los códigos de estado, el JSON y las redirecciones de la pasarela.
- **Sistema en el navegador:** `tests/system/` levanta la app en un hilo con datos en memoria, pasarela y mail simulados y el reloj fijo, y la recorre con Playwright usando el Microsoft Edge instalado, en una pantalla de celular (390×844). Cubren las 4 pruebas de usuario de la US y los flujos principales (D46).

Trazabilidad con los criterios de aceptación

Criterio de aceptación	Casos que lo cubren
CA1 Fecha, cantidad, edad y pase	EDA, PAS, FEC-11/12, CAN-08/09, ORD-02/08, SRV-03/10, API-01/09, SIS-07
CA2 Fecha actual o futura	FEC-01 a FEC-07, API-02
CA3 Mail de confirmación	MAI, MAIL, SRV-01/04/12, API-19, SIS-01/05
CA4 Pago con tarjeta (pasarela simulada)	PAG-07 a PAG-10, TAR, PSM, SRV-02 a SRV-08, API-05 y API-12 a API-20, SIS-01 y SIS-06
CA5 Parque abierto	DIA, FEC-10/15, ORD-09, API-07, SIS-03
CA6 Forma de pago	PAG-01 a PAG-05, API-06, SIS-02/05
CA7 Máximo 10 entradas	CAN, ORD-09, SRV-09, API-08, SIS-04
CA8 Informar cantidad y fecha	ORD-05 a ORD-07, SRV-14, API-04, API-19, SIS-01
CA9 Solo usuarios registrados	USR-01 a USR-03, API-21